

Improving diagnostics for sender expressions

Authors:

[Eric Niebler](#)

Date:

February 29, 2024

Source:

[GitHub](#)

Issue tracking:

[GitHub](#)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

LEWG

Synopsis

This paper aims to improve the user experience of the sender framework by moving the diagnosis of invalid sender expression earlier, when the expression is constructed, rather than later when it is connected to a receiver. A trivial change to the sender adaptor algorithms makes it possible for the majority of sender expressions to be type-checked early.

Executive Summary

Below are the specific changes this paper proposes in order to make early type-checking of sender expressions possible:

1. Define a “non-dependent sender” to be one whose completions are knowable without an environment.
2. Extend the awaitable helper concepts to support querying a type whether it is awaitable in an arbitrary coroutine (without knowing the promise type). For example, anything that implements the awaiter interface (`await_ready`, `await_suspend`, `await_resume`) is awaitable in any coroutine, and should function as a non-dependent sender.
3. Add support for calling `get_completion_signatures` without an environment argument.

4. Change the definition of the `completion_signatures_of_t` alias template to support querying a sender's non-dependent signatures, if such exist.
5. Require the sender adaptor algorithms to preserve the “non-dependent sender” property wherever possible.
6. Add “Mandates:” paragraphs to the sender adaptor algorithms to require them to hard-error when passed non-dependent senders that fail type-checking.

Problem Description

Type-checking a sender expression involves computing its completion signatures. In the general case, a sender's completion signatures may depend on the receiver's execution environment. For example, the sender:

```
read(get_stop_token)
```

... when connected to a receiver `rcvr` and started, will fetch the stop token from the receiver's environment and then pass it back to the receiver, as follows:

```
auto st = get_stop_token(get_env(rcvr));  
set_value(move(rcvr), move(st));
```

Without an execution environment, the sender `read(get_stop_token)` doesn't know how it will complete.

The type of the environment is known rather late, when the sender is connected to a receiver. This is often far from where the sender expression was constructed. If there are type errors in a sender expression, those errors will be diagnosed far from where the error was made, which makes it harder to know the source of the problem.

It would be far preferable to issue diagnostics while *constructing* the sender rather than waiting until it is connected to a receiver.

Non-dependent senders

The majority of senders have completions that don't depend on the receiver's environment. Consider `just(42)` – it will complete with the integer 42 no matter what receiver it is connected to. If a so-called “non-dependent” sender advertised itself as such, then sender algorithms could eagerly type-check the non-dependent senders they are passed, giving immediate feedback to the developer.

For example, this expression should be immediately rejected:

```
just(42) | then([](int* p) { return *p; })
```

The `then` algorithm can reject `just(42)` and the above lambda because the arguments don't match: an integer cannot be passed to a function expecting an `int*`. The `then` algorithm can do that type-checking only when it knows the input sender is non-dependent. It couldn't, for example, do any type-checking if the input sender were `read(get_stop_token)` instead of `just(42)`.

And in fact, some senders *do* advertise themselves as non-dependent, although P2300 does not currently do anything with that extra information. A sender can declare its completions signatures with a nested type alias, as follows:

```
template <class T>
struct just_sender {
    T value;

    using completion_signatures =
        std::execution::completion_signatures<
            std::execution::set_value_t(T)
        >;

    // ...
};
```

Senders whose completions depend on the execution environment cannot declare their completion signatures this way. Instead, they must define a `get_completion_signatures` customization that takes the environment as an argument.

We can use this extra bit of information to define a `non_dependent_sender` concept as follows:

```
template <class Sndr>
concept non_dependent_sender =
    sender<Sndr> &&
    requires {
        typename remove_cvref_t<Sndr>::completion_signatures;
    };
```

A sender algorithm can use this concept to conditionally dispatch to code that does eager type-checking.

Suggested Solution

The authors suggest that this notion of non-dependent senders be given fuller treatment in P2300. Conditionally defining the nested typedef in generic sender adaptors – which may adapt either dependent or non-dependent senders – is awkward and verbose. We suggest instead to support calling `get_completion_signatures` either with *or without* an execution environment. This makes it easier for authors of sender adaptors to preserve the “non-dependent” property of the senders it wraps.

We suggest that a similar change be made to the `completion_signatures_of_t` alias template. When instantiated with only a sender type, it should compute the non-dependent completion signatures, or be ill-formed.

Design Considerations

Why have two ways for non-dependent senders to publish their completion signatures?

The addition of support for a customization of `get_completion_signatures` that does not take an environment obviates the need to support the use of a nested `::completion_signatures` alias. In a class, this:

```
auto get_completion_signatures() ->
    std::execution::completion_signatures<
        std::execution::set_value_t(T)
    >;
```

... works just as well as this:

```
using completion_signatures =
    std::execution::completion_signatures<
        std::execution::set_value_t(T)
    >;
```

Without a doubt, we could simplify the design by dropping support for the latter. This paper suggests retaining it, though. For something like the `just_sender`, providing type metadata with an alias is more idiomatic and less surprising, in the author's opinion, than defining a function and putting the metadata in the return type. That is the case for keeping the typename `Sndr::completion_signatures` form.

The case for adding the `sndr.get_completion_signatures()` form is that it makes it simpler for sender adaptors such as `then_sender` to preserve the “non-dependent” property of the senders it adapts. For instance, one could define `then_sender` like:

```
template <class Sndr, class Fun>
struct then_sender {
    Sndr sndr_;
    Fun fun_;

    template <class... Env>
    auto get_completion_signatures(const Env&... env) const
        -> some-computed-type;

    //....
};
```

... and with this one member function support both dependent and non-dependent senders while preserving the “non-dependent-ness” of the adapted sender.

Proposed Wording

[Editorial note: The wording in this section assumes the adoption of [P2855R1](#). -- end note]

Change [async.ops]/13 as follows:

13. A completion signature is a function type that describes a completion operation. An asynchronous operation has a finite set of possible completion signatures corresponding to the completion operations that the asynchronous operation potentially evaluates ([basic.def.odr]). For a completion function set, receiver rcvr, and pack of arguments args, let c be the completion operation set(rcvr, args...), and let F be the function type decltype(auto(set))(decltype(args)...). A completion signature Sig is associated with c if and only if MATCHING-SIG(Sig, F) is true ([exec.general]). Together, a sender type and an environment type Env determine the set of completion signatures of an asynchronous operation that results from connecting the sender with a receiver that has an environment of type Env. [Note: The type of the receiver does not affect an asynchronous operation's completion signatures, only the type of the receiver's environment. -- end note] A sender type whose completion signatures are knowable independent of an execution environment is known as a *non-dependent sender*.

Change [exec.syn] as follows:

```
...  
template<class Sndr, class... Env =empty_env>  
    concept sender_in = see below;  
...  
  
template<class Sndr, class... Env =empty_env>  
    requires sender_in<Sndr, Env...>  
using completion_signatures_of_t = call-result-  
t<get_completion_signatures_t, Sndr, Env...>;  
...
```

Change [exec.snd.concepts] as follows:

```
template<class Sndr, class... Env =empty_env>  
    concept sender_in =  
        sender<Sndr> &&  
        (sizeof...(Env) <= 1)  
        (queryable<Env> &&...) &&  
        requires (Sndr&& sndr, Env&&... env) {  
            { get_completion_signatures(  
                std::forward<Sndr>(sndr), std::forward<Env>(env)...)  
            }  
        }  
        -> valid-completion-signatures;  
};
```

[Editorial note: this subtly changes the meaning of sender_in<Sndr>. Before the change, it tests whether a type is a sender when used specifically with the environment empty_env. After the change, it tests whether a type is a non-dependent sender. This is a stronger assertion to make about the type; it says that this type is a sender *regardless of the environment*. One can still get the old behavior with sender_in<Sndr, empty_env>. - end note]

Change [exec.awaitables] as follows:

1. The sender concepts recognize awaitables as senders. For this clause ([exec]), an ***awaitable*** is an expression that would be well-formed as the operand of a `co_await` expression within a given context.
2. For a subexpression `c`, let `GET-AWAITER(c, p)` be expression-equivalent to the series of transformations and conversions applied to `c` as the operand of an *await-expression* in a coroutine, resulting in lvalue `e` as described by [expr.await]/3.2-4, where `p` is an lvalue referring to the coroutine's promise type, `Promise`. This includes the invocation of the promise type's `await_transform` member if any, the invocation of the operator `co_await` picked by overload resolution if any, and any necessary implicit conversions and materializations. Let `GET-AWAITER(c)` be expression-equivalent to `GET-AWAITER(c, q)` where `q` is an lvalue of an unspecified empty class type *none-such* that lacks an `await_transform` member, and where `coroutine_handle<none-such>` behaves as `coroutine_handle<void>`.
3. Let *is-awaitable* be the following exposition-only concept:

```
template<class T>
concept await_suspend_result = see below;

template<class A, class... Promise>
concept isawaiter = // exposition only
    requires (A& a, coroutine_handle<Promise...> h) {
        a.await_ready() ? 1 : 0;
        { a.await_suspend(h) } -> await_suspend_result;
        a.await_resume();
    };

template<class C, class... Promise>
concept isawaitable =
    requires (C (*fc)() noexcept, Promise&... p) {
        { GET-AWAITER(fc(), p...) } -> is-
awaiter<Promise...>;
    };

```

`await_suspend_result<T>` is true if and only if one of the following is true:

- `T` is `void`, or
- `T` is `bool`, or
- `T` is a specialization of `coroutine_handle`.

4. For a subexpression `c` such that `decltype((c))` is type `C`, and an lvalue `p` of type `Promise`, `await_result_type<C, Promise>` denotes the type `decltype(GET-AWAITER(c, p).await_resume())`, and `await_result_type<C>` denotes the type `decltype(GET-AWAITER(c).await_resume())`.

Change [exec.getcomplsigs] as follows:

1. `get_completion_signatures` is a customization point object. Let `sndr` be an expression such that `decltype((sndr))` is `Sndr`, ~~and let `env` be an expression such that `decltype((env))` is `Env`.~~ Then `get_completion_aignatures(sndr)` is expression-equivalent to:
 1. `remove_cvref_t<Sndr>::completion_signatures{}` if that expression is well-formed,

2. Otherwise, `decltype(sndr.get_completion_signatures()){} if that expression is well-formed,`

3. Otherwise, if `is-awaitable<Sndr>` is true, then:

```
completion_signatures<
    SET-VALUE-SIG(await-result-type<Sndr>),
    // see [exec.snd.concepts]
    set_error_t(exception_ptr),
    set_stopped_t()>{}

```

4. Otherwise, `get_completion_signatures(sndr)` is ill-formed.

2. Let `env` be an expression such that `decltype((env))` is `Env`. Then `get_completion_signatures(sndr, env)` is expression-equivalent to:

1. `remove_cvref_t<Sndr>::completion_signatures{} if that expression is well-formed,`

2. `Otherwise, decltype(sndr.get_completion_signatures(env)){} if that expression is well-formed,`

2. ~~`Otherwise, remove_cvref_t<Sndr>::completion_signatures{} if that expression is well-formed,`~~

3. Otherwise, if `is-awaitable<Sndr, env-promise<Env>>` is true, then:

```
completion_signatures<
    SET-VALUE-SIG(await-result-type<Sndr,
env-promise<Env>>), // see [exec.snd.concepts]
    set_error_t(exception_ptr),
    set_stopped_t()>{}

```

4. Otherwise, `get_completion_signatures(sndr, env)` is ill-formed.

3. If `get_completion_signatures(sndr)` is well-formed and its type denotes a specialization of the `completion_signatures` class template, then `Sndr` is a non-dependent sender type ([`async.ops`]).

4. Given a pack of subexpressions `e`, the expression `get_completion_signatures(e...)` is ill-formed if `sizeof...(e)` is less than 1 or greater than 2.

5. If `completion_signatures_of_t<Sndr>` and `completion_signatures_of_t<Sndr, Env>` are both well-formed, they shall denote the same set of completion signatures, disregarding the order of signatures and rvalue reference qualification of arguments.

6. Let `rcvr` be an rvalue receiver of type `Rcvr`....

To [exec.adapt.general], add a paragraph (8) as follows:

8. Unless otherwise specified, an adaptor whose child senders are all non-dependent ([`async.ops`]) is itself non-dependent. This requirement applies to any function that is selected by the implementation of the sender adaptor.

Change [`exec.then`] as follows:

2. The names `then`, `upon_error`, and `upon_stopped` denote customization point objects. For `then`, `upon_error`, and `upon_stopped`, let `set-cpo` be `set_value`, `set_error`, and `set_stopped` respectively. Let the expression `then-cpo` be one of `then`, `upon_error`, or `upon_stopped`. For subexpressions `sndr` and `f`, let `Sndr` be `decltype((sndr))` and let `F` be the decayed type of `f`. If `Sndr` does not satisfy `sender`, or `F` does not satisfy `movable-value`, `then-cpo(sndr, f)` is ill-formed.
3. Otherwise, let `invoke-result` be an alias template such that `invoke-result<Ts...>` denotes the type `invoke_result_t<F, Ts...>`. If `sender_in<Sndr>` is true and `gather-signatures<tag_t<set-cpo>, completion_signatures_of_t<Sndr>, invoke-result, type-list>` is ill-formed, the program is ill-formed.
4. Otherwise, the expression `then-cpo(sndr, f)` is expression-equivalent to:.....
5. ~~For `then`, `upon_error`, and `upon_stopped`, let `set-cpo` be `set_value`, `set_error`, and `set_stopped` respectively.~~

The exposition-only class template `impls-for` ([`exec.snd.general`]) is specialized for `then-cpo` as follows:....

Change [`exec.let`] by inserting a new paragraph between (4) and (5) as follows:

5. Let `invoke-result` be an alias template such that `invoke-result<Ts...>` denotes the type `invoke_result_t<F, Ts...>`. If `sender_in<Sndr>` is true and `gather-signatures<tag_t<set-cpo>, completion_signatures_of_t<Sndr>, invoke-result, type-list>` is ill-formed, the program is ill-formed.

Change [`exec.bulk`] by inserting a new paragraph between (3) and (4) as follows:

5. Let `invoke-result` be an alias template such that `invoke-result<Ts...>` denotes the type `invoke_result_t<F, Shape, Ts...>`. If `sender_in<Sndr>` is true and `gather-signatures<tag_t<set-cpo>, completion_signatures_of_t<Sndr>, invoke-result, type-list>` is ill-formed, the program is ill-formed.

Acknowledgments

We owe our thanks to Ville Voutilainen who first noticed that most sender expressions could be type-checked eagerly but are not by P2300R8.